AIRA — System Design Document

Rajesh Kumar Rama Reddy INFO 7375 — Prompt Engineering and AI

---

1. Problem Statement

University academic integrity policies are legal-weight documents written in natural language. They define what students may and may not do, what constitutes a violation, and what enforcement actions follow. But most policies were written before generative AI existed, and many still are. The result: clauses that prohibit "AI tools" without defining them, enforcement mechanisms that assume analog cheating, and scope statements that create exploitable gaps.

Manual policy review is slow, inconsistent, and does not scale. A reviewer reading fifty clauses will apply different thresholds on clause 48 than on clause 3. No standard taxonomy exists for categorizing policy risk. No mechanism exists to flag when two clauses in the same document contradict each other.

AIRA solves this. It ingests any university academic integrity PDF, applies a structured eight-category risk taxonomy to every clause, cites the exact source text for every finding, verifies those citations for faithfulness, and detects contradictions between clause pairs — all at consistent quality, in under sixty seconds, at a cost below $0.50 per document.

---

2. User and Business Needs

2.1 User Needs

User	Need	How AIRA Addresses It
Policy administrator	Identify which specific clauses contain exploitable loopholes, with exact source text	Clause-level risk annotations with `cited_text` field and `reasoning` explanation
Student	Understand what the policy actually permits or prohibits	AIUsageLoophole category flags ambiguous AI boundaries; full clause text shown

User	Need	How AIRA Addresses It
	regarding AI use before submitting work	
Policy author	Ranked list of risk findings to prioritize rewrites	Risk distribution heatmap + flagged clause table sorted by category
Instructor	Quick audit of a new institution's policy before teaching	Upload tab: drag PDF, receive full report in 30–60 seconds

2.2 Academic / Business Need

AIRA is the course final project for INFO7375: Prompt Engineering and AI. It must demonstrate four core competencies: prompt engineering, retrieval-augmented generation (RAG), fine-tuning, and evaluation. The system must be publicly deployed and operable by a professor without setup.

---

3. System Architecture

3.1 Deployment Topology

```
┌──────────────────────────────────────────────────────────────┐
│ User's browser
│ https://academic-integrity-risk-analyzer.vercel.app
│
│ React SPA (Vercel CDN)
│ ├── About tab      — static, no API calls
│ ├── Demo tab       — GET /demo or /demo/harvard
│ ├── Upload tab     — POST /analyze
│ └── Evaluation tab — GET /evaluation + /finetune/*
└──────────────────────────────────────────────────────────────┘
                        │ HTTPS REST
                        ▼
┌──────────────────────────────────────────────────────────────┐
│ FastAPI backend (Render free tier)
│ https://aira-api.onrender.com
│
│ On startup: loads chroma_db/ into memory
│ Endpoints: /health /demo /demo/harvard
│            /analyze /evaluation /finetune/*
│
│ Runtime calls:
│ ├── OpenAI API (classifier, contradiction, alignment)
│ └── ChromaDB in-process (pre-built, no network calls)
```

3.2 Component Diagram

```
PDF upload
    │
    ▼

 ingestion.py      PDF → List[Clause]
                   pdfplumber extraction
                   noise filter (TOC, nav)
                   sentence-boundary split
                   min 8 words per clause

       │
       ▼

 embeddings.py     Clause → ChromaDB query
                   text-embedding-3-small
                   pre-built index (chroma_db/)
                   get_all_embeddings() for contradiction

       │
       ▼

classifier.py      Clause → RiskAnnotation
                   GPT-4o, temp=0, json_object
                   8-category taxonomy enforcement
                   cited_text required in every call
                   rate-limit retry (4 attempts)

       │
       ├──────────────────────────┐
       ▼                          ▼

 alignment.py          contradiction.py

 cited_text            cosine sim pairs
 verbatim              (threshold 0.4)
 check +               GPT-4o pairwise
 cosine sim            check, top 30
 fallback              pairs only

       │                          │
       └──────────────────────────┘
                    ▼

 AuditReport (models.py)
 source_doc, total_clauses
 flagged_count, annotations
 contradictions
 faithfulness_score
 overall_risk_rating
 risk_distribution

                    │
                    ▼
          FastAPI /analyze
          → JSON response
          → React frontend
```

4. Component Design

4.1 ingestion.py

Responsibility: Convert a PDF file into a list of clean, filtered Clause objects.

Key decisions:

- `pdfplumber` chosen over PyMuPDF for cleaner text extraction from policy PDFs (better whitespace handling, fewer encoding artifacts)
- Noise filter uses regex patterns to drop: page numbers, table of contents entries, single-word lines, lines with >30% non-ASCII characters, navigation elements
- Sentence-boundary splitting (`.`, `?`, `!`) rather than fixed-length chunking — preserves the logical unit of a policy clause
- Minimum 8-word filter eliminates headers, section titles, and fragment artifacts
- Output: `Clause(id, text, source_doc, page_num, position_index, word_count)`

Tested on: MIT (454 clauses), Harvard (29), NEU (66)

4.2 embeddings.py

Responsibility: Load the pre-built ChromaDB index and expose query interfaces.

Key decisions:

- Index pre-built locally via `build_index.py` and committed to the repo — no embedding API calls at runtime (see §6.2 for rationale)
- One ChromaDB collection per source document (slug of filename)
- `get_all_embeddings()` returns both Clause objects and raw embedding vectors — used by `contradiction.py` to compute cosine similarity across all clause pairs without additional API calls
- `index_loaded()` boolean used by `/health` endpoint to verify startup succeeded

4.3 classifier.py

Responsibility: Classify each Clause into exactly one of eight risk categories.

System prompt design:

- Provides full definition of each category with distinguishing examples
- Requires `cited_text`: the exact substring of the clause that justifies the label
- Requires `reasoning`: chain-of-thought explanation before the label
- Conservative flagging guidance: target 40–60% flag rate; not every clause is risky
- `response_format: json_object` — prevents free-form output, enforces schema

Model selection:

- Primary: `gpt-4o` (90.2% accuracy on 51-clause ground truth)
- Runtime default in `main.py`: `ft:gpt-4o-mini-2024-07-18:personal:aira:DXBLUexI` (80.4% accuracy, ~20× cheaper — acceptable for live upload use)
- Fine-tuned model trained on 86 examples (69 train / 17 val) derived from ground truth and diversity-filtered synthetic clauses

Reliability: 4-attempt backoff on `RateLimitError`; invalid category names default to `"None"` rather than raising

4.4 alignment.py

Responsibility: Verify that each annotation's `cited_text` is faithful to the source clause.
Scoring logic:

1. Check if `cited_text` is a verbatim substring of `clause_text` → score `1.0`
2. If not: compute cosine similarity between embeddings of both strings → use as score
3. `None`-category annotations always score `1.0` (no citation to verify)

Threshold: `0.65` — annotations below this score are flagged `low_confidence_flag: True`
Result on NEU ground truth: 100% faithfulness — all citations are exact verbatim substrings. Cosine fallback is a safety net; it has not triggered in practice.

4.5 contradiction.py

Responsibility: Detect pairs of clauses in the same document that contradict each other.

Two-stage approach:

1. **Candidate generation:** Compute cosine similarity across all clause pairs using pre-loaded embeddings (no new API calls). Retain pairs with similarity > 0.4, sorted descending, capped at top 30.
2. **LLM verification:** For each candidate pair, ask GPT-4o: "Do these clauses contradict each other? If so, does scope explain the conflict?" Only report contradictions
   where `is_contradiction=true` AND `scope_explains_conflict=false`.

Contradiction types: `PermissionConflict`, `PunishmentConflict`, `ScopeOverlap`, `PolicyGap`, `None`
Cost control: 30-pair cap prevents unbounded API cost on large documents. At MIT's 454 clauses, the full pairwise space is ~103,000 pairs. Semantic pre-filtering reduces this to 30 GPT-4o calls.

4.6 models.py (Pydantic schemas)

```
Clause            — id, text, source_doc, page_num, position_index,
word_count
RiskAnnotation    — clause_id, clause_text, risk_category,
secondary_category,
                    reasoning, cited_text, confidence, low_confidence_flag
Contradiction     — clause_a, clause_b, contradiction_type, explanation,
                    scope_explains_conflict
AuditReport       — source_doc, total_clauses, flagged_count, annotations,
                    contradictions, faithfulness_score,
overall_risk_rating,
                    risk_distribution
SyntheticClause   — text, category, is_adversarial, human_accepted
EvaluationResult  — accuracy, precision_per_category, recall_per_category,
                    confusion_matrix, faithfulness_score,
hallucination_rate,
                    run_timestamp
```

4.7 main.py (FastAPI)

Endpoints:

Method	Path	Purpose	Notes
`GET`	`/health`	Liveness check + index status	Hit before demo to wake Render
`GET`	`/demo`	Pre-computed NEU analysis	Zero API calls; loads `demo_output.json`
`GET`	`/demo/harvard`	Pre-computed Harvard analysis	Zero API calls
`POST`	`/analyze`	Live pipeline for uploaded PDF	200-clause cap; runs full pipeline
`GET`	`/evaluation`	Latest evaluation results	Reads most recent `evaluation/results/*.json`
`GET`	`/finetune/status`	Fine-tune job metadata	Static JSON of completed job
`GET`	`/finetune/compare`	Base vs. fine-tuned accuracy	Static comparison JSON

Startup: ChromaDB index loaded once via FastAPI `lifespan` context manager. CORS configured for Vercel frontend origin.

---

5. Risk Taxonomy

Eight mutually exclusive primary labels. Every clause receives exactly one.

Category	Definition
`Ambiguity`	Language admits multiple legitimate interpretations
`UndefinedTerm`	A term central to the clause is used without definition
`EnforcementGap`	No mechanism exists to detect or act on a violation
`ScopeConflict`	The clause applies inconsistently across contexts or populations
`AuthorityConflict`	Unclear who holds enforcement authority
`AIUsageLoophole`	AI use is permitted or prohibited without a clear, enforceable boundary
`CircularDefinition`	A definition references itself or its own definiendum
`None`	No identifiable risk

Secondary label allowed when genuine overlap exists
(e.g., `AIUsageLoophole` primary + `UndefinedTerm` secondary when the AI term itself is undefined).

---

6. Design Decisions and Rationale

6.1 Pre-build ChromaDB vs. Runtime Embedding

Decision: Build the vector index locally once (`build_index.py`) and commit `chroma_db/` to the repository. No embedding API calls at inference time.
Rationale: `text-embedding-3-small` costs per call. A 454-clause document (MIT) would incur ~454 embedding calls on every upload. Pre-building means zero marginal embedding cost for the demo documents. For uploaded PDFs, embeddings are computed only for contradiction detection, not for the entire document.
Trade-off accepted: The committed index is static. Adding new reference documents requires re-running `build_index.py` locally. Acceptable for a course project where the document set is fixed.

6.2 Structured JSON Output vs. Open Generation

Decision: All GPT-4o classification calls use `response_format: json_object` with a defined schema (`risk_category`, `reasoning`, `cited_text`, `confidence`). Temperature fixed at 0.
Rationale: Open-ended generation produces category names that do not match the taxonomy (e.g., "Vague Language" instead of "Ambiguity"). This breaks the downstream confusion matrix and evaluation pipeline. `json_object` enforcement eliminates this failure mode entirely. Temperature 0 ensures deterministic classification — the same clause produces the same label on re-run.
Trade-off accepted: Structured output is slightly more expensive than completion tokens alone. The accuracy and reliability gain is worth it.

6.3 Fine-Tuned Fallback vs. Always GPT-4o

Decision: Deploy `ft:gpt-4o-mini-2024-07-18:personal:aira:DXBLUexI` as the runtime default for live uploads.
Rationale: GPT-4o costs approximately $2.50/1M input tokens. gpt-4o-mini costs approximately $0.15/1M — roughly 17× cheaper. Fine-tuning on 86 examples raises mini's accuracy from its untuned baseline to 80.4%, which is 5.9 percentage points below GPT-4o's 90.2% on the same ground truth. For a live upload tool serving untrusted PDFs of unknown length, this tradeoff is acceptable: the tool remains useful while the per-upload cost stays below $0.05.

The Demo tab uses pre-computed output (see §6.5) — the fine-tuned model is never called during a demo, only during live uploads.

6.4 200-Clause Cap per Upload

Decision: Return HTTP 400 with an informative message if an uploaded PDF produces more than 200 clauses.

Rationale: MIT's academic handbook produces 454 clauses. At GPT-4o pricing with the fine-tuned mini, 454 clauses cost approximately $0.07 per upload. At GPT-4o base pricing, the same document costs approximately $4.54 — more than the entire monthly free tier budget. The 200-clause cap bounds the worst case at approximately $0.03 per upload and prevents a single large document from exhausting API quota.

6.5 Pre-Computed Demo Tab

Decision: The Demo tab fetches a static JSON file (`demo_output.json`) rather than calling the pipeline at render time. Zero API calls.
Rationale: Render's free tier sleeps instances after 15 minutes of inactivity. Cold start latency is 20–40 seconds.

6.6 Cosine Similarity Threshold for Contradiction Candidates

Decision: Retain clause pairs with cosine similarity ≥ 0.4 for GPT-4o contradiction verification. Cap candidate set at 30 pairs.

Rationale: A threshold of 0.4 captures semantically related clauses (which are the only pairs likely to contradict) without requiring LLM verification of every combination.

7. Data Flow — End to End

```
User uploads PDF
        │
        ▼
POST /analyze
        │
        ▼
ingestion.extract_clauses(pdf_bytes)
  → List[Clause]          [~1–5 seconds for 50-clause doc]
        │
        ├─ len(clauses) > 200? → HTTP 400
        │
        ▼
embeddings.get_all_embeddings(collection)
  → List[Clause], List[embedding]    [from pre-built index — instant]
        │
        ▼
classifier.classify_batch(clauses)
  → List[RiskAnnotation]    [~25–50 seconds at 1 call/clause]
        │
        ├──────────────────────────────┐
        ▼                              ▼
alignment.compute_faithfulness()   contradiction.detect_all()
  → float [0.0–1.0]               → List[Contradiction]
  [instant — no API calls]        [~3–10 seconds, 30 pairs max]
        │                              │
        └──────────────────────────────┘
                       ▼
              AuditReport assembled
              + overall_risk_rating computed
                       │
                       ▼
              JSON response → React frontend
```

Typical end-to-end latency: 30–60 seconds for a 50-clause document. **Primary bottleneck:** Sequential GPT-4o calls in `classify_batch()`.

8. API Specification

POST /analyze

Request: `multipart/form-data` with field `file` (PDF, max 10 MB)
Response: `AuditReport` JSON

```
{
  "source_doc": "policy.pdf",
  "total_clauses": 51,
  "flagged_count": 21,
  "annotations": [
    {
      "clause_id": "uuid",
      "clause_text": "...",
      "risk_category": "EnforcementGap",
      "secondary_category": null,
      "reasoning": "...",
      "cited_text": "...",
      "confidence": 0.87,
      "low_confidence_flag": false
    }
  ],
  "contradictions": [
    {
      "clause_a": { "clause_id": "...", "clause_text": "..." },
      "clause_b": { "clause_id": "...", "clause_text": "..." },
      "contradiction_type": "PermissionConflict",
      "explanation": "...",
      "scope_explains_conflict": false
    }
  ],
  "faithfulness_score": 1.0,
  "overall_risk_rating": "Medium",
  "risk_distribution": {
    "Ambiguity": 5,
    "EnforcementGap": 8,
    "AIUsageLoophole": 3
  }
}
```

Error responses:

- `400` — file is not a PDF, exceeds 10 MB, or exceeds 200-clause cap
- `500` — OpenAI API failure (propagates `detail` message)

GET /health

Response: `{"status": "ok", "index_loaded": true}`

GET /demo, GET /demo/harvard

Response: Pre-computed `AuditReport` JSON (identical schema to `/analyze`)

9. Non-Functional Requirements and Constraints

Constraint	Value	Source
Maximum file size	10 MB	Frontend validation + backend check
Maximum clauses per upload	200	Cost control
Classification temperature	0	Determinism requirement
Synthetic generation temperature	0.7	Diversity requirement
All GPT-4o calls	`response_format: json_object`	Schema enforcement
Every finding	Must include `cited_text`	No unsupported claims
Demo tab	Zero API calls	Cold-start reliability
API key storage	Render environment variables only	Never committed to repo
ChromaDB index	Pre-built, committed to repo	No runtime embedding cost

---

10. Failure Modes and Mitigation

Failure	Likelihood	Impact	Mitigation
Render cold start (20–40s delay)	High — free tier sleeps	Demo fails in front of professor	Hit `/health` 5 min before demo; Demo tab uses pre-computed output
OpenAI rate limit during `classify_batch`	Medium — long documents	Partial classification	4-attempt exponential backoff in `classify_clause`

Failure	Likelihood	Impact	Mitigation
ChromaDB index fails to load at startup	Low	All endpoints fail	`index_loaded` flag in `/health`; manual restart
PDF with non-standard encoding	Medium	Extraction produces garbage clauses	Noise filter + 8-word minimum removes most artifacts
GPT-4o returns invalid category name	Low	Breaks evaluation pipeline	Defaults to `"None"` rather than raising; logged
Uploaded PDF exceeds 200 clauses	Possible for handbooks	Cost overrun	Hard 400 rejection with clear user message
Fine-tuned model deprecation	Future risk	Live uploads degrade	Base model fallback available in `classifier.py`

11. Evaluation Methodology

11.1 Ground Truth

51 clauses from the NEU Academic Integrity Policy, manually annotated with the correct risk category. Annotations recorded in `data/ground_truth.json` with fields: `text`, `source_doc`, `page_num`, `risk_category`.

11.2 Evaluation Pipeline

`backend/evaluator.py` runs the AIRA classifier on all 51 ground truth clauses, computes accuracy, per-category precision/recall, confusion matrix, and faithfulness score. Results saved to `evaluation/results/run_{timestamp}.json`.

11.3 Synthetic Data

56 clauses generated by `backend/synthetic.py`:

- 35 clean clauses (one per category, representative examples)
- 21 adversarial clauses (designed to appear clean while containing a risk)

- Diversity filter: cosine similarity threshold 0.85 rejects near-duplicate generations
- `human_accepted` field defaults `False` — requires human review gate before adversarial clauses are used to compute hallucination rate

11.4 Results

Metric	Result
Overall accuracy	**90.2%** (46/51)
Faithfulness	**100%** — all citations exact verbatim substrings
Hallucination rate	**0.0%** on adversarial synthetic set
Best category	AIUsageLoophole: 100% precision / 100% recall
Worst category	ScopeConflict: 0% precision / 0% recall
Ambiguity recall	60% — misses 4 of 10 true cases

11.5 Baseline Comparison

Condition	Accuracy	Notes
Vanilla GPT-4o (no structure)	58.8%	Category names only; collapses to Ambiguity/None
Fine-tuned gpt-4o-mini	80.4%	AIRA structured prompt; ~20× cheaper
GPT-4o base (AIRA prompt)	86.3%	Full structured prompt; base model
AIRA full pipeline	**90.2%**	Full structured prompt; best evaluation run

Key finding: Structured taxonomy prompting adds +27.5 percentage points over vanilla GPT-4o (58.8% → 86.3%). Fine-tuning adds a further +4.1 points in some configurations. The taxonomy definitions — not the model size — drive the majority of the accuracy gain.

12. Boondoggle Score

The boondoggle score separates what Claude automated from what the human decided. Five supervisory capacities are scored 0–5 per dimension.

12.1 What Claude Built

- All Python module implementations (ingestion, embeddings, classifier, alignment, contradiction, evaluator, synthetic, models, main)
- React component implementations (Scorecard, ClauseTable, ContradictionView, RiskHeatmap, EvaluationDashboard)
- React view implementations (DemoView, UploadView, App.jsx with routing)
- Fine-tuning data formatting and job submission code
- Baseline evaluation script (baseline.py)
- requirements.txt, package.json configuration

12.2 What the Human Decided

Decision	Human Judgment Required
Problem definition	"Policy ambiguity" is the right problem; not summarization, not QA
Taxonomy design	Eight specific categories; their definitions and boundaries
Conservative flagging guidance	"40–60% target" — prevents over-flagging
Citation enforcement	Every finding must cite source text — no unsupported claims
ChromaDB pre-build strategy	Recognized runtime embedding as a cost trap
Clause cap at 200	Calculated the MIT worst-case cost; set the limit
Demo pre-computation	Identified cold-start as a demo-killing failure mode

Decision	Human Judgment Required
Fine-tuning decision	Decided 5.9% accuracy loss is acceptable for 20× cost reduction
Ground truth annotation	51 clauses manually labeled — no delegation possible
Faithfulness threshold	Set at 0.65 based on calibration of what "supported" means
Evaluation design	Decided to test vanilla baseline to isolate the prompt contribution
Honest limitation disclosure	ScopeConflict failure, narrow ground truth, synthetic data gate

12.3 Scores by Supervisory Capacity

Capacity	Score	Evidence
Plausibility Auditing	4/5	Caught ScopeConflict blind spot; identified that 86.3% fine-tune baseline ≠ vanilla baseline; recognized cold-start as undocumented risk
Problem Formulation	5/5	Defined policy auditing as distinct from summarization; designed the 8-category taxonomy before any code was written
Tool Orchestration	4/5	Sequenced build correctly; chose pre-build strategy; designed evaluation before running it; ran vanilla baseline proactively
Interpretive Judgment	4/5	Interpreted 80.4% as "cheaper, not better"; recognized that 100% faithfulness is an easy condition; bounded generalizability claims
Executive Integration	4/5	Held four competencies (prompting, RAG, fine-tuning, eval) in one coherent system; cold-start mitigation designed before deployment

Overall Boondoggle Score: 21/25

The minions built the rocket. Gru designed the mission, wrote the taxonomy, set the cost controls, identified the failure modes, and decided what 80.4% means in context. That gap — between what was built and what was decided — is where all the intellectual work lives.

---

13. Known Limitations

1. **ScopeConflict is undetected** — 0% precision and recall. This category requires reasoning about whether a clause applies differently in different contexts, which demands cross-clause understanding. Single-clause classification cannot supply this.

2. **Narrow ground truth** — 51 clauses from one institution (NEU). Performance on Harvard AI Guidelines (26 clauses) is qualitatively consistent but not formally evaluated. Claims about accuracy should be bounded to NEU-style policies.

3. **Faithfulness metric is easy** — 100% faithfulness holds because all cited texts are verbatim substrings. The metric does not test paraphrased or hallucinated citations. A more adversarial faithfulness test would require a separate LLM judge.

4. **Synthetic adversarial data unreviewed** — `human_accepted` defaults `False`. The 0.0% hallucination rate is computed only on clauses where `human_accepted=True`. If no clauses were manually accepted, the hallucination rate is trivially zero (empty denominator) and not meaningful.
5. **Sequential classification** — `classify_batch()` calls the LLM one clause at a time. Parallelizing with `asyncio` or thread pools would reduce end-to-end latency by approximately 5–10×. Not implemented in v1.